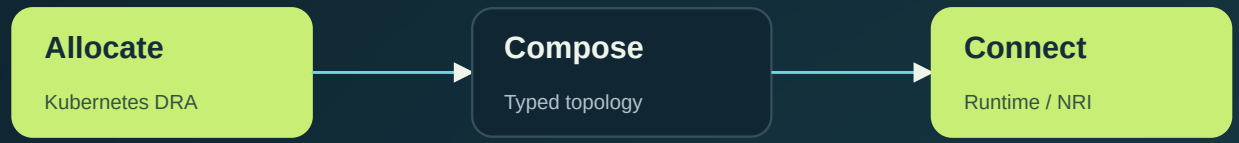
# A network is more than an interface.

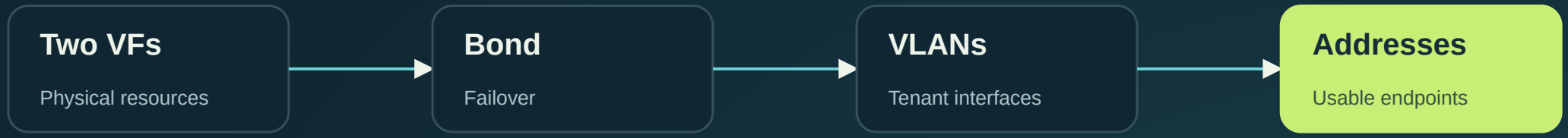
## DRA Multi-Network




SECONDARY NETWORKS. EXPLICIT OWNERSHIP. ONE LIFECYCLE.

# Allocate the hardware. Describe the whole network.

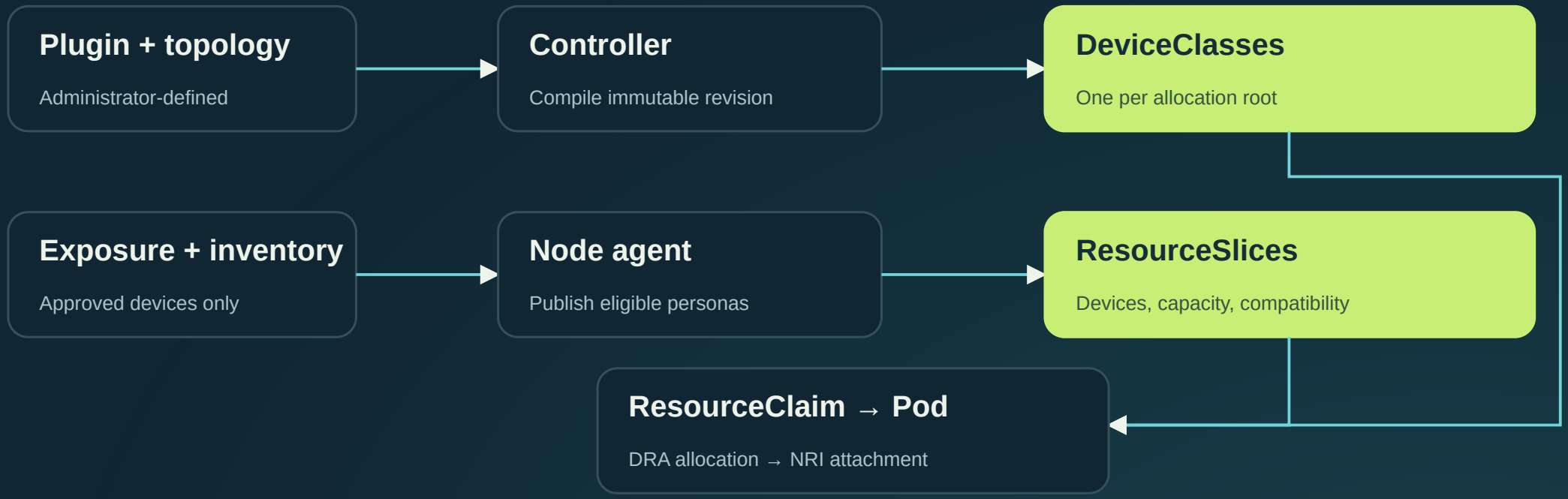
A VF is only the beginning of the connection.




> Resource allocation + network composition + runtime attachment

# Two inputs. One allocated contract.

Administrator intent meets policy-approved node inventory.



# Five CRs. Clear ownership.

Four cluster-scoped contracts; one namespaced observation.

ADMIN

## NetworkPlugin

Schema, typed ports, allowed effects, cleanup contract.

ADMIN

## NetworkTopology

Allocation roots + operation steps in a dependency graph.

CONTROLLER

## NetworkTopologyRevision

Immutable topology, locked plugin digests and execution order.

ADMIN

## DeviceExposurePolicy

Which hardware, personas and mutations are approved.

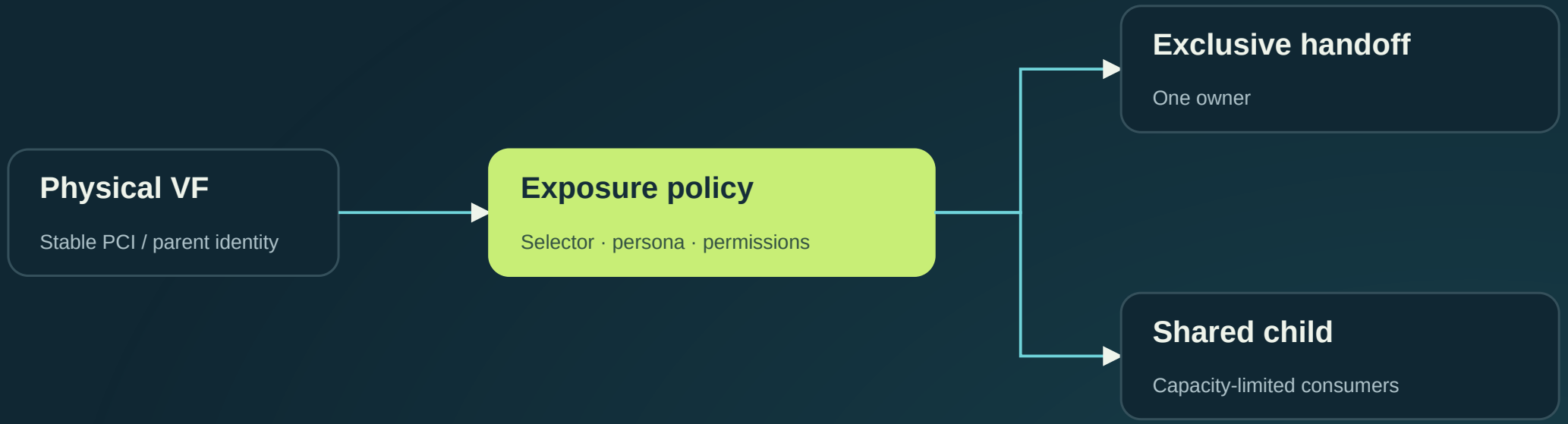
NODE · NAMESPACED

## NetworkAttachmentStatus

Attachment identity, phase, outputs and errors.

# Expose personas.
# Protect physical ownership.

Multiple views of a device still share one physical resource.




> Selection ≠ mutation authority. Both are checked.

# Advertise alternatives. Grant mutations.

Two policies expose choices within the same PF family; allocation enforces exclusivity.

HOST PF · EXCLUSIVE HANDOFF

```yaml
apiVersion: networking.dra-multi-network.schseba.github.io/v1alpha1
kind: DeviceExposurePolicy
metadata: {name: secondary-host}
spec:
  priority: 100
  selector:
    kind: PF
    pciAddresses: ["0000:29:00.0"]
  action: Expose
  exposure:
    persona: kernel
    plugin: host-cni
    mode: kernel
    effectPermissions:
    - linux/link-move/v1
    - linux/link-configure/v1
```

CHILD VFs · EXCLUSIVE HANDOFF PER VF

```yaml
apiVersion: networking.dra-multi-network.schseba.github.io/v1alpha1
kind: DeviceExposurePolicy
metadata: {name: secondary-vfs}
spec:
  priority: 100
  selector:
    kind: VF
    parentPCIAddresses: ["0000:29:00.0"]
  action: Expose
  exposure:
    persona: kernel
    plugin: sriov-cni
    mode: kernel
    effectPermissions:
    - linux/link-move/v1
    - linux/link-configure/v1
```

**SELECTOR**

PF PCI identity vs. VFs belonging to that PF.

**PERSONA + PLUGIN + MODE**

Which approved device profile is advertised.

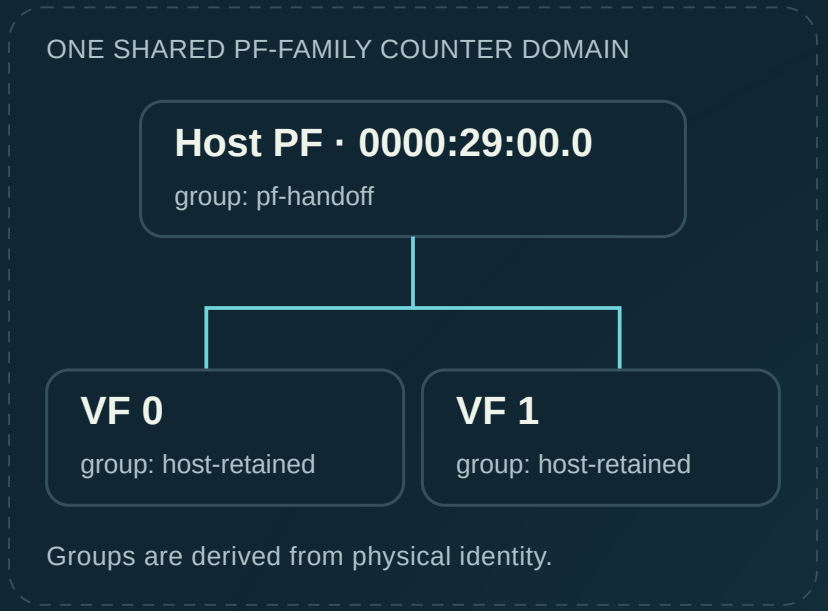
**EFFECT PERMISSIONS**

Which native mutations policy authorizes.

Example PCI address: adapt to dedicated secondary hardware. Management devices remain protected.

# A PF and its VFs are mutually exclusive.

Exclusive PF handoff conflicts with its child VFs. Different VFs can coexist.




**Host PF handoff + VF 0** — BLOCKED

Same family; groups do not intersect.

**VF 0 + VF 1** — COMPATIBLE*

Same family; both are host-retained.

**VF active → request host PF handoff** — BLOCKED

Conflict works in both allocation directions.

* Subject to per-VF ownership, available capacity and other selectors.

Scheduler checks ResourceSlice compatibility; the node rechecks live ownership before mutation.

KUBERNETES ALPHA · v1.37 · DISABLED BY DEFAULT

## DRADeviceCompatibilityGroups

Groups on `device.consumesCounters[]`

Requires `DRAPartitionableDevices` too.

# Six profiles. Two compatibility domains.

Core derives domains from physical identity. Groups constrain allocation; grants authorize mutations.

| PROFILE | INTERFACE DOMAIN | INTERFACE GROUP | PF-FAMILY GROUP |
|---|---|---|---|
| **Host PF · exclusive** | PF PCI | `kernel-handoff` | `pf-handoff` |
| **SR-IOV VF · exclusive** | VF PCI | `kernel-handoff` | `host-retained` |
| **macvlan on PF** | PF PCI | `macvlan` | `host-retained` |
| **ipvlan on PF** | PF PCI | `ipvlan` | `host-retained` |
| **macvlan on VF** | VF PCI | `macvlan` | `host-retained` |
| **ipvlan on VF** | VF PCI | `ipvlan` | `host-retained` |

PF-FAMILY CHECK

### PF handoff excludes its family

`pf-handoff` conflicts with every `host-retained` profile on that PF.

SAME INTERFACE CHECK

### Handoff / macvlan / ipvlan

Different groups block mixing on one parent. Same-mode children share within capacity.

DIFFERENT INTERFACES

### PF children + VF use can coexist

Different interface domains; matching `host-retained` family group.

gRPC TARGET / PLUGIN DECLARES

Ownership + compatible mode + mutation scope

→

CORE BINDS AND PUBLISHES

Physical domains + groups + authoritative capacity

No plugin-selected counter domains or duplicate capacity. Core rechecks live ownership before native mutation.

# A plugin declares a contract.

Typed ports and approved effects make composition checkable.

BOND-CNI · NETWORKPLUGIN CR EXCERPT

```yaml
apiVersion: networking.dra-multi-network.schseba.github.io/v1alpha1
kind: NetworkPlugin
metadata: {name: bond-cni}
spec:
  kind: Operation
  implementation: {type: Builtin, name: bond-cni, version: v1}
  inputs:
    members:
      type: KernelInterface/v1
      access: ExclusiveMaster
      minItems: 2
      maxItems: 8
  outputs:
    interface:
      type: KernelInterface/v1
      ownership: Exclusive
      minItems: 1
      maxItems: 1
  allowedEffects:
    - linux/create-bond/v1
    - linux/attach-master/v1
    - linux/link-configure/v1
  stateContract: {version: v1, cleanup: RecordedEffects}
```

Config schema and Ethernet link types omitted for readability.

01 / GRANTED INPUTS

## 2–8 owned members

Topology references supply Ethernet interfaces. The framework grants exclusive master access.

02 / TYPED OUTPUTS

## One bond interface

A `KernelInterface/v1` output becomes the next step's input — for example, a VLAN parent.

03 / EFFECT CEILING

## Explicit native operations

The planner may request only declared effects. Policy and topology budgets further constrain execution.

# Bounded effects. Recorded cleanup.

The planner requests operations. The framework executes and restores them.

BOND-CNI · EFFECTS + STATE CONTRACT

```
allowedEffects:
  - linux/create-bond/v1
  - linux/attach-master/v1
  - linux/link-configure/v1

stateContract:
  version: v1
  cleanup: RecordedEffects
```

/v1 versions each effect opcode.
stateContract.version versions the state / cleanup contract.

`linux/create-bond/v1`

**Create the bond interface**

`linux/attach-master/v1`

**Attach the granted member interfaces**

`linux/link-configure/v1`

**Apply MTU and link up / down state**

RECORDEDEFFECTS / CLEANUP

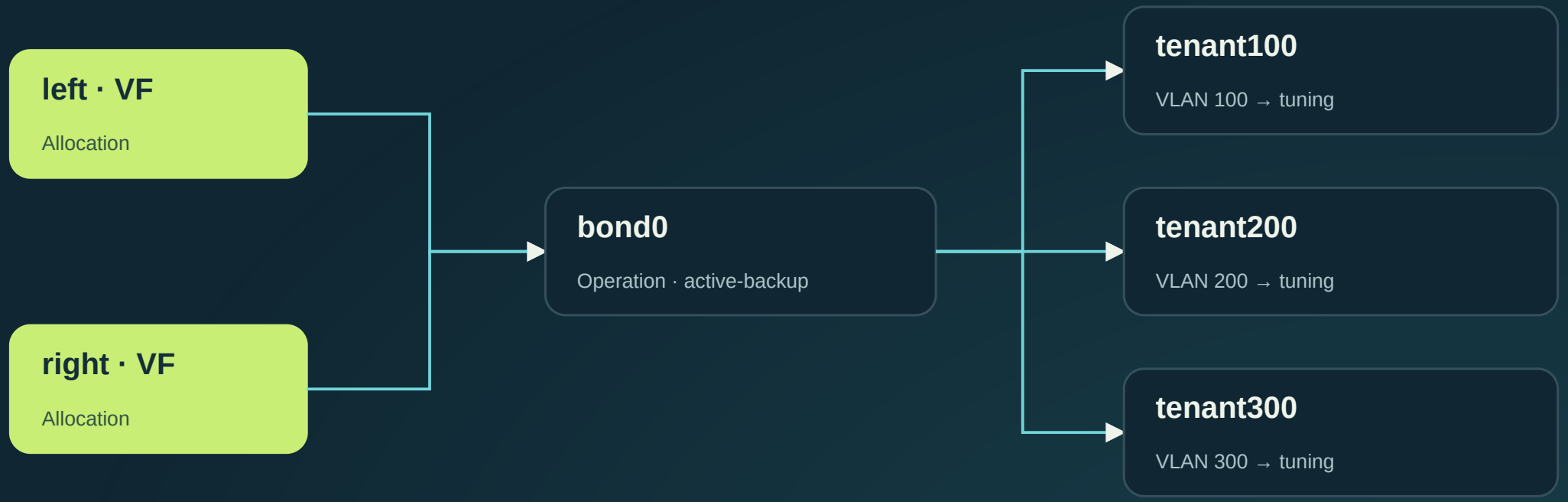
## Restore the saved execution

Reverse order · saved before-images · identity and live-state checks

Record intent → Execute effects → Restore original state

# Compose a network as a graph.

Example: two allocation roots → active-backup bond → three VLANs.




> Two allocated devices. One complete topology.

# The YAML defines the dependency graph.

Allocation steps request resources. Operation steps compose their outputs.

DUAL-VF-BOND-VLAN · NETWORKTOPOLOGY EXCERPT

```yaml
apiVersion: networking.dra-multi-network.schseba.github.io/v1alpha1
kind: NetworkTopology
metadata: {name: dual-vf-bond-vlan}
spec:
  effectBudget:
    allow:
    - linux/create-bond/v1
    - linux/attach-master/v1
    - linux/create-vlan/v1
  steps:
  - {id: left, kind: Allocation, pluginRef: {name: sriov-cni}}
  - {id: right, kind: Allocation, pluginRef: {name: sriov-cni}}
  - id: uplink
    kind: Operation
    pluginRef: {name: bond-cni}
    inputs:
      members: [{from: left.interface}, {from: right.interface}]
    config: {ifName: bond0, mode: active-backup}
  - id: tenant100
    kind: Operation
    pluginRef: {name: vlan-cni}
    inputs: {parent: {from: uplink.interface}}
    config: {ifName: tenant100, vlanId: 100}
```

Abbreviated: selectors, required config fields, other effects / VLANs / tuning omitted.

01 / EFFECT BUDGET

## Bound the whole graph

`effectBudget.allow` caps the requested mutations across every step. Plugin permissions and device policy also apply.

02 / ALLOCATION ROOTS

## left + right → two classes

Each `Allocation` root produces a DeviceClass. Selectors constrain hardware; config describes the requested interface.

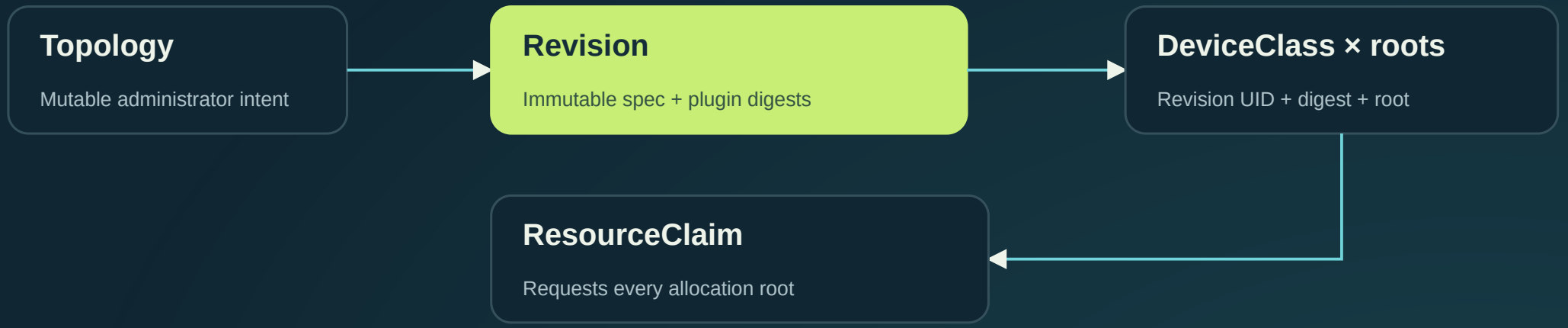
03 / OPERATION + REFERENCES

## Wire outputs into inputs

`left.interface` + `right.interface` feed the bond. `uplink.interface` feeds the VLAN. Config sets behavior; references determine execution order.

# Freeze intent before execution.

Existing claims keep their locked contract when administrators edit a topology.



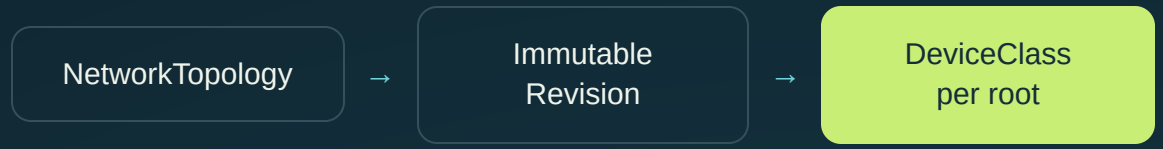
# The claim binds resources to a revision.

Class-origin configuration carries the locked topology into node execution.

RESOURCECLAIM · TWO ROOT REQUESTS

```yaml
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata:
  name: dual-vf-network
  namespace: multi-network-example
spec:
  devices:
    requests:
    - name: primary
      exactly:
        deviceClassName: LEFT_CLASS
        allocationMode: ExactCount
        count: 1
    - name: backup
      exactly:
        deviceClassName: RIGHT_CLASS
        allocationMode: ExactCount
        count: 1
```

LEFT_CLASS / RIGHT_CLASS = generated names from topology.status.rootClasses.
Example workload uses a ResourceClaimTemplate to create this claim per Pod.




DEVICECLASS → RESOURCECLAIM

## primary → left · backup → right

Classes embed revision name / UID / digest,
root step and plugin contract digest.

DRA allocates one VF per request from ResourceSlices.
The claim must cover every root of the same revision.

01 / DRA NODEPREPARE

### Verify the locked root bindings

Allocated identities + class configuration resolve the saved revision, then preparation is journaled.

02 / NRI CONNECTNETWORK

### Execute the complete graph

Granted VFs → bond → VLANs → tuning.
Before consuming containers start.

# Bound class usage per namespace.

ResourceQuota limits requested devices per class at ResourceClaim admission.

TEAM-A · ALLOW UP TO FOUR COMPLETE TWO-VF CLAIMS

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: bond-network-budget
  namespace: team-a
spec:
  hard:
    LEFT_CLASS.deviceclass.resource.k8s.io/devices: "4"
    RIGHT_CLASS.deviceclass.resource.k8s.io/devices: "4"
```

TEAM-B · BLOCK NEW CLAIMS FOR THESE ROOT CLASSES

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: bond-network-budget
  namespace: team-b
spec:
  hard:
    LEFT_CLASS.deviceclass.resource.k8s.io/devices: "0"
    RIGHT_CLASS.deviceclass.resource.k8s.io/devices: "0"
```

BUDGET BOTH ROOTS

## 4 left + 4 right

Up to four complete claims request eight VFs total.
Quota counts requested devices, not available hardware.

NAMESPACE ADMISSION

## 0 devices → new claims denied

Applies to manual and template-generated claims.
Users must not be able to modify the quota.

**Missing class quota ≠ denied.** Use admission class allowlists for default-deny. Update quotas / rules for new revision class names.

LEFT_CLASS / RIGHT_CLASS are placeholders for the generated names in topology.status.rootClasses.

# Connect before the application starts.

DRA provides allocation lifecycle; NRI provides the Pod sandbox lifecycle.

01 · KUBELET / DRA

## NodePrepare

Verify allocation

Journal hardware preparation

02 · RUNTIME / NRI

## ConnectNetwork

Execute topology in sandbox

Verify before container start

03 · RUNTIME / NRI

## RemoveNetwork

Restore connection effects

Reverse dependency order

04 · KUBELET / DRA

## NodeUnprepare

Restore prepared hardware

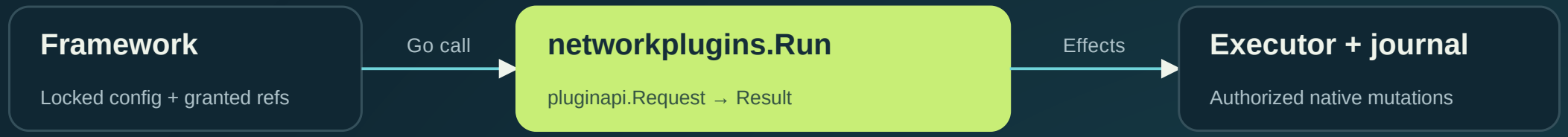
Release durable ownership

Primary CNI → normal Pod network

Secondary attachment → this project

# Internal planners.
# A transport-neutral boundary.

Serializable Go requests and results already separate planning from execution.




No CNI binary execution. No external planner server.

> The lifecycle contract exists. The remote transport is future work.

# One contract. Multiple network shapes.

All eleven built-in planners use the same request / result boundary.

01

## Allocate interfaces

host-cni · sriov-cni

02

## Create shared children

macvlan-cni · ipvlan-cni

03

## Compose & tune

bond-cni · vlan-cni · tuning-cni

04

## Attach representors

accelerated-bridge-cni · ovs-cni

05

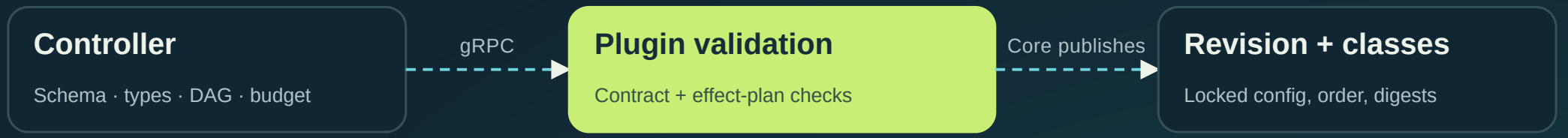
## Connect addressing

ipam · address-cni

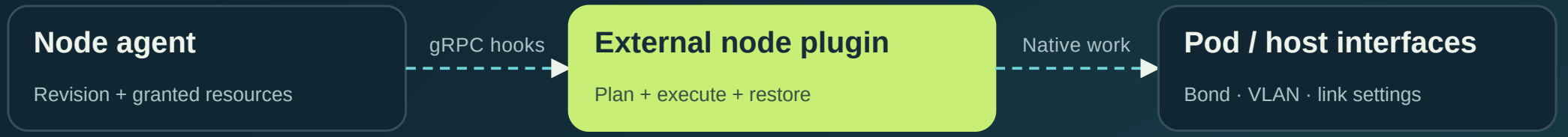
# Two phases. One plugin boundary.

Vendors add planning and native execution without changing core network mechanisms.

01 / TOPOLOGY APPLIED · COMPILE & VALIDATE · NO INTERFACE CHANGES




02 / ALLOCATED POD · LIFECYCLE HOOKS · INTERFACE CHANGES




NodePrepare · ConnectNetwork · RemoveNetwork · NodeUnprepare

> Vendor network mechanisms live in plugins; core DRA, policy and lifecycle coordination stay shared.

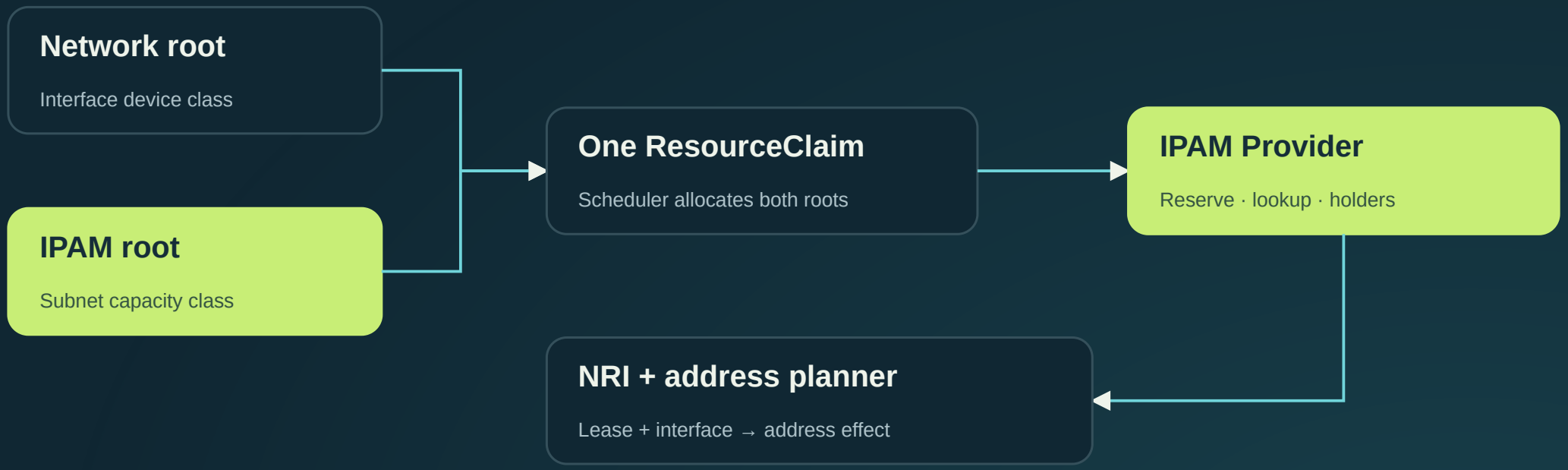
# “gRPC” means three different things.

Only the kubelet boundary uses gRPC in the current POC.

| BOUNDARY | TODAY | REVISED TARGET |
| --- | --- | --- |
| **Kubelet → DRA driver** | **gRPC · implemented** | Framework keeps registration |
| **Framework → network plugin** | In-process Go call | gRPC planning + native execution |
| **Framework → IPAM provider** | Go Provider interface | Stateful gRPC service |

# Allocate capacity. Reserve an address.

The planner is pure. The provider owns durable reservations.




> Scheduler-visible scarcity + durable address ownership

# IPAM is a DRA allocation root.

The topology declares the root. A ResourceClaim triggers Kubernetes allocation.

MACVLAN-IPAM · NETWORKTOPOLOGY EXCERPT

```yaml
apiVersion: networking.dra-multi-network.schseba.github.io/v1alpha1
kind: NetworkTopology
metadata: {name: macvlan-ipam}
spec:
  effectBudget:
    allow: [linux/create-macvlan/v1,
            linux/link-configure/v1, linux/address-configure/v1]
  steps:
  - id: endpoint
    kind: Allocation
    pluginRef: {name: macvlan-cni}
    config: {ifName: secondary0, mtu: 1500, up: true,
             mode: bridge, linkType: ethernet}
  - id: address
    kind: Allocation
    pluginRef: {name: ipam}
    config: {network: blue, subnet: 192.0.2.0/29}
  - id: configure-address
    kind: Operation
    pluginRef: {name: address-cni}
    inputs:
      interface: {from: endpoint.interface}
      lease: {from: address.lease}
    config: {}
```

Hardware / persona selectors omitted; full example targets approved secondary devices.

01 / ADDRESS ALLOCATION ROOT

## pluginRef: ipam

The approved plugin descriptor sets `allocationDriver` to `ipam.schseba.github.io`. Core generates an address-root DeviceClass.

02 / REQUEST THROUGH DRA

## Subnet device → address share

IPAM publishes the subnet in ResourceSlices. The claim requests one address-capacity unit; Kubernetes allocates a share, then the provider reserves its concrete IP.

03 / COMPOSE THE OUTPUTS

## Interface + lease → configured IP

`endpoint.interface` and `address.lease` feed `address-cni`. This Operation creates no additional DeviceClass or DRA request.

# The address class selects a subnet.

Generated from the topology's address root — not a class for each concrete IP.

CONTROLLER-GENERATED DEVICECLASS · EXCERPT

```yaml
apiVersion: resource.k8s.io/v1
kind: DeviceClass
metadata: {name: IPAM_CLASS}
spec:
  selectors:
  - cel:
      expression: >
        device.driver == "ipam.schseba.github.io"
        && device.attributes["ipam.schseba.github.io"].network == "blue"
        && device.attributes["ipam.schseba.github.io"].subnet == "192.0.2.0/29"
  config:
  - opaque:
      driver: dra-multi-network.schseba.github.io
      parameters:
        kind: TopologyRootConfiguration
        rootStep: address
        # Revision identity + digests omitted here.
  - opaque:
      driver: ipam.schseba.github.io
      parameters:
        apiVersion: ipam.schseba.github.io/v1alpha1
        kind: IPAMAllocationConfiguration
        network: blue
        subnet: 192.0.2.0/29
        addresses: 1
```

Omitted: shared / ready selector checks, root binding apiVersion, revision UID / digests and metadata.

01 / SELECT THE ADDRESS POOL

## blue · 192.0.2.0/29

CEL selects the IPAM driver and approved subnet. The complete class also requires a shared, allocation-ready device.

02 / LOCK THE TOPOLOGY ROOT

## Class-origin configuration

The network driver receives the revision binding and `rootStep: address`. Users cannot substitute claim-supplied configuration.

03 / RESERVE ONE ADDRESS

## IPAM backend configuration

The IPAM driver receives network, subnet and `addresses: 1`. Capacity lives in ResourceSlices; the provider reserves the concrete IP during preparation.

# One claim requests interface + address.

Two allocation roots. One complete network before containers start.

RESOURCECLAIM · MACVLAN + ONE ADDRESS

```yaml
apiVersion: resource.k8s.io/v1
kind: ResourceClaim
metadata: {name: macvlan-ipam, namespace: ipam-example}
spec:
  devices:
    requests:
    - name: endpoint
      exactly:
        deviceClassName: ENDPOINT_CLASS
        allocationMode: ExactCount
        count: 1
        capacity:
          requests:
            dra-multi-network.schseba.github.io/children: "1"
    - name: address
      exactly:
        deviceClassName: IPAM_CLASS
        allocationMode: ExactCount
        count: 1
        capacity:
          requests:
            ipam.schseba.github.io/addresses: "1"
```

Class names are placeholders from topology.status.rootClasses.
The shipped example generates this claim from a ResourceClaimTemplate.

01 / ENDPOINT_CLASS

### One macvlan child

`count: 1` selects one shared parent device; `children: "1"` consumes one child-capacity unit.

02 / IPAM_CLASS

### One address-capacity unit

`count: 1` selects one subnet device; `addresses: "1"` consumes one IP unit. It does not select or configure a specific IP.

03 / RUNTIME JOIN

### Interface + lease → address

Provider reserves the IP; `address-cni` uses `endpoint.interface` + `address.lease`. Exhausted subnet capacity keeps the claim Pending.

# Where feedback can help.

Review the ownership model before standardizing the transport.

01

## API boundaries

Are topology, exposure and attachment responsibilities clear?

02

## Lifecycle & composition

Which runtime semantics and failure cases should be required?

03

## Plugin interoperability

What belongs in a versioned planner protocol and conformance suite?

Allocate with DRA. Compose with typed contracts. Connect with NRI.

# The implementation map.

Useful anchors for questions and follow-up.

CR definitions
`pkg/api/networking/v1alpha1/types.go`

Planner contract
`pkg/pluginapi/api.go`

Internal planners
`pkg/networkplugins/builtin.go`

Binding + validation
`pkg/topology/plan.go`

Native execution
`pkg/effects/`

Stateful IPAM contract
`pkg/ipam/provider.go`

Vendor plugin gRPC target
`docs/grpc-plugins.md`

Recorded validation
`docs/validation.md`